

JFTSE Tournament Mode

Complete 64→16 lifecycle, multi-client match orchestration, durable progression, archives, restart recovery, and exactly-once prize settlement for the selected Basic format.

Result: the unmodified Fantasy Tennis client lists and enters tournaments, renders qualifying/final state, runs assigned two-client matches through relay, advances the durable bracket, and completes a native championship whose inventory prize persists exactly once across restart.

Repository	sstokic-tgm/JFTSE
Branch	feature/tournament-mode
Continuation base	516a3073e627d13256c5964459d48af266b88feb
Implementation	da6e435e47744318fb16998478dcdd3e1d1b2ea7
Faulty commit removed	fcdeb89a94bb17337f8b8f499b79c5892370004e
Revert commit	e817af5a
Validation date	12 August 2026

Contents

Executive result	4
Documentation used as the game specification	4
Scope boundary	5
Branch and faulty commit removal	6
First-principles reverse-engineering method	6
Client identity	7
Recovered packet family	7
0x26B0 fixed tournament arrays	7
0x26C1 final bracket definition	8
0x26C3 match records and the decisive sentinel	8
Additional static client findings	9
Server implementation	10
Packet schema	10
Handlers	10
State model	10
Lifecycle, scheduling, and creation policy	11
Bracket policy and progression	11
Room, relay, spectators, and teardown	11
Archives and prize settlement	12
Red/green development record	12
Red 1: no tournament handlers	12
Red 2: list existed, detail response was missing	13
Red 3: 0x26AE omitted mandatory success metadata	14
Red 4: final bracket request unhandled	15
Red 5: definition accepted, automatic match request unhandled	15
Red 6: correct count, invalid record state	16
Green: exact packet contract and real-client flow	17

Real-client end-to-end visual evidence	18
1. Client reached the local authentication server	18
2. Test credentials were entered	19
3. Test character and local channel were selected	19
4. Client connected to the local game server	20
5. Tournament list rendered	20
6. Tournament detail and winner rewards rendered	21
7. Apply confirmation	23
8. Apply success	24
9. Applied state persisted across detail refreshes	25
10. Cancel confirmation and success	26
11. Nonparticipant state restored	28
12. Final bracket visibly rendered	28
13. Zoom in and out controls worked	29
14. Round reward control opened and closed	30
15. Back navigation returned to tournament details	31
Native two-client tournament match continuation	32
16. Assigned players joined a locked tournament room	32
17. Ready/start and relay handoff succeeded	32
18. Completed native match advanced the durable bracket	34
19. Native championship settled the durable first-place prize once	34
Network and runtime evidence	36
Primary captures	37
First accepted bracket exchange	37
Reproduction	37
Java validation	37
PCAP extraction	38
Runtime client parser verification	38
Tests added	38
Known limitations and next evidence required	39
Conclusion	40

Executive result

This continuation turns the original client-validated protocol slice into a durable tournament engine while retaining the recovered wire contract:

- the real client lists a scheduled `JFTSE Open Cup`;
- tournament metadata, dates, mode, entry type, and winner rewards render;
- a player can apply, see the applied state, cancel, and see the nonparticipant state restored;
- the enabled final-tournament-bracket control opens a real bracket screen;
- the client renders a 16-entry final bracket and continuously polls its 15 internal match records;
- the bracket zoom, round-reward panel, close, and back controls work without disconnecting the client;
- qualifying is modeled as 64 entrants, 32 first-round matches, and 16 qualifier matches shown as eight six-row pages;
- completed matches advance winners through qualifying and the 16→8→4→2→1 final bracket;
- tournament definitions, enrollment, matches, runtime bindings, and settlements are persisted in MariaDB;
- a scheduler creates a recurring default cup, advances due lifecycle states, aborts underfilled stages, and recovers stale runtime bindings after restart;
- assigned players create and join locked `T#<tournamentId>` rooms, connect through relay, play a real Basic match, and return to the room;
- ordinary room positions above the four player positions remain available to spectators, without treating spectators as relay-readiness participants;
- the winner receives the configured inventory product through an idempotent settlement record;
- all recovered request/response packets and the added lifecycle policies are covered by byte-level, service, concurrency, scheduler, room, relay, and MariaDB tests.

The distinction between **retail facts** and **emulator policy** is essential. The client and wiki prove lifecycle/status vocabulary, a six-row qualifying-page view, a 16-entry final view, and the recovered packet family. They do not prove retail seeding, timer policy, server-side administration, live spectator admission, archive transport, or prize-award transactions. The branch implements conservative server policy for those gaps; it does not present that policy as recovered original-server behavior. There is no high-confidence native create/admin request in this client build, so tournament creation is server-side.

No client binary patch was required. The downloaded archive was preserved, while a runtime copy was configured for the loopback services and instrumented only during protocol diagnosis.

Documentation used as the game specification

The JFTSE wiki was treated as first-party project documentation:

1. [JFTSE Roadmap](#) explicitly lists Tournament Mode under both reverse engineering and planned emulator work. It also warns that packet identification alone is insufficient: surrounding structures and value effects require trial and error.
2. [Game Modes](#) describes registration into available tournaments and a 64-player bracket in the original overall mode.
3. [Packet Structure](#) defines the fixed eight-byte header, little-endian fields, packet ID, and payload length.

4. [Packet Schema \(.packet\) Format](#) defines generated CMSG parsing, server packet serialization, UTF-16LE strings, Windows FILETIME dates, repeated fields, and `[len = N]` fixed arrays.
5. [Database Schema & Cheatsheet](#) documents the existing schema and configuration model. It does not document tournament tables; the added tables are explicitly emulator-owned persistence policy.
6. [All Pages](#) was reviewed to identify all relevant protocol, schema, mode, and roadmap material.

The wiki establishes the product intent and common wire conventions. Exact Tournament Mode bytes were derived from the client, not guessed from prose.

Scope boundary

Capability	Result	Evidence basis
Tournament list	Implemented	Real-client request/response and visible row
Detail and rewards	Implemented	Real-client <code>0x26AD/AE</code> and <code>0x26BE/BF</code> exchange
Apply and cancel	Durable	Native UI plus serializable MariaDB mutation tests
Final bracket	Implemented	Visible 16-entry bracket, parser <code>c3_ok</code> trace
Match-state polling	Implemented	Repeated <code>0x26C2/C3</code> every approximately 10 seconds
Bracket zoom/reward/back UI	Validated	Real-client screenshots
Qualifying pages	Implemented as policy	Static client RE proves six visible rows; server exposes pages 0–7
64→16 qualifying	Implemented as policy	32 first-round plus 16 qualifier matches; deterministic service tests
Final progression	Implemented	16→8→4→2→1 winner advancement with concurrent-completion tests
Tournament room and relay	Implemented	Two unmodified clients created/joined <code>T#2</code> , started, and registered on one relay session
Spectators	Ordinary room support	Positions <code>>3</code> ; running-match admission remains unsupported/unproven
Archives	Durable service API	Finished/canceled records persist; no recovered native archive packet endpoint
Prize settlement	Implemented as policy	Native final inventory delta plus unique settlement, rollback, and exactly-once tests
Scheduling and restart	Durable	10-second scheduler, recurring default cup, stale-binding packaged-JAR recovery
Tournament creation	Server-side	Validated 64/16 creation API plus idempotent recurring default creation
Authoritative scoring	Not implemented	Match completion still trusts existing client-reported score/end packets

Branch and faulty commit removal

The requested faulty commit was removed with a dedicated revert:

```
e817af5a Revert "fix types in pet packets; adjust S2CRoomPlayerListInformationPacket to include pet; room player cant change slots with pet"
```

```
This reverts commit fcdeb89a94bb17337f8b8f499b79c5892370004e.
```

After implementation, the latest development branch was merged. Development contained a follow-up that depended on the faulty BattleMon room-packet shape; the merge conflicts were resolved in favor of the explicit revert, so the removed payload was not silently reintroduced. The latest unrelated development fixes are otherwise present.

First-principles reverse-engineering method

The implementation followed a strict evidence loop:



All client/server tests used loopback addresses. No public game-server endpoint was configured.

Client identity

```
FantaTennis.7z
SHA-256 c19ca21b8e2ab091953b2f631e48853b6477400f4d7000682ac7440f9994f12e

FantaTennis.exe
SHA-256 5477f0827acae66976403aec2e9ebffeb4fa28da1fedae5f9541ec25e336c31
Size 3,801,088 bytes
```

Recovered packet family

Every field below is little-endian after the standard eight-byte packet header.

ID	Direction	Purpose	Recovered payload
0x26AD	C→S	Open/join tournament detail	tournamentId:int32
0x26AE	S→C	Detail metadata	status:int8 ; on success, metadata through bracket size
0x26AF	C→S	List page	page:int32
0x26B0	S→C	Tournament list	count-prefixed tournaments
0x26B1	C→S	Apply	tournamentId:int32
0x26B2	S→C	Apply result	status:int8
0x26B3	C→S	Cancel	tournamentId:int32
0x26B4	S→C	Cancel result	status:int8
0x26BE	C→S	Participation/detail state	tournamentId:int32
0x26BF	S→C	Participation/detail state	result plus success-only player state block
0x26C0	C→S	Bracket definition	tournamentId:int32, bracketType:uint8, page:uint8
0x26C1	S→C	Bracket definition	status plus selectors and fixed-width entry rows
0x26C2	C→S	Bracket match-state poll	tournamentId:int32, bracketType:uint8, matchIndex:uint8
0x26C3	S→C	Bracket match-state result	status plus selectors, count:int16 , two-byte records

The server accepts qualifying selector `bracketType = 0` on pages 0-7 and final selector `bracketType = 1` on page 0. Each qualifying page contains six persisted matches/rows; the final contains 16 entries and 15 match records. Other selectors receive a terminal nonzero response instead of bytes from a different stage.

0x26B0 fixed tournament arrays

The list structure contains:

- exactly five reward records, each two `int32` values;
- exactly 16 pairs in bracket array A;
- exactly 16 pairs in bracket array B.

The client consumes these arrays directly, without a count before each array. The pre-existing generator wrote a count for every repeated server field, even when the schema declared `[len = N]`. That shifted every subsequent byte and prevented a valid tournament record.

`FTPacketGen` now treats fixed repeated fields symmetrically:

1. no count is written on the wire;
2. null or wrong-size values fail fast;
3. primitive and composite repeated fields follow the same rule;
4. normal repeated fields retain their old count-prefixed behavior.

The golden tests assert both the complete tournament byte order and wrong-cardinality failures.

0x26C1 final bracket definition

Accepted success payload:

```
status      int8      0
tournamentId int32     1
bracketType  uint8     1 (final)
page        uint8     0
unknown     uint8     0
entryCount  int16     16
entries[16]:
  first      12 bytes   fixed UTF-16LE, null-padded
  second     12 bytes   fixed UTF-16LE, null-padded
  third      12 bytes   fixed UTF-16LE, null-padded
```

Payload size:

```
1 + 4 + 1 + 1 + 1 + 2 + (16 × 36) = 586 bytes
586-byte payload + 8-byte header = 594-byte TCP packet
```

0x26C3 match records and the decisive sentinel

The client derives its structural expectations from the final-bracket metadata:

```
entries = 16
rounds  = 5
nodes   = 31
matches = nodes - entries = 15
```

The response layout is:

```
status      int8      0
tournamentId int32     1
bracketType  uint8     1
matchIndex   uint8     0
matchCount   int16     15
matches[15]:
  result      int8      -1 (0xFF, empty/unresolved sentinel)
  state       uint8     0
```

Payload size:

```
1 + 4 + 1 + 1 + 2 + (15 × 2) = 39 bytes
39-byte payload + 8-byte header = 47-byte TCP packet
```

Sending 15 records of 00 00 passed the count comparison but failed a later client state validator. Runtime uprobes narrowed the rejection to the branch after the count check. Static analysis of that branch identified a signed sentinel domain; changing only the first state byte to 0xFF produced the client success branch and visible bracket.

The preserved success trace states:

```
c3_count:   loaded=1 complete=0 entries=16 rounds=5 nodes=31 expected=15
c3_compare: got=0xf expected=0xf
c3_ok:      loaded=1 complete=0 expected=15
```

Subsequent polls show complete=1 and continue through c3_ok.

Additional static client findings

The continuation rechecked the exact executable hash above and recovered these additional high-confidence client facts:

- lifecycle values 0..8 map to PREPARE, APPLY, PREPARE_QUALIFYING, QUALIFYING, PREPARE_FINAL, FINAL, FINISHED, SUSPENDED, and CANCELED;
- personal values 0..4 map to not applied, applied, entered, dropped, and winner;
- C1 type 0 selects a qualifying model by page index, and the qualifying UI performs exactly six row lookups per visible page;
- a row is structurally three fixed 12-byte UTF-16 fields, but their retail semantic names remain unknown;
- C6 contains only tournamentId:int32 and is reachable only for myState == 2; it is participant-only and therefore is not evidence of spectator entry;
- C7 is a tagged server update capable of replacing tournament/player state; static evidence does not prove it is a direct C6 response;
- B5/B6 and B7/B8 have parsers in a separate qualifying-screen controller. Their exact layouts are recovered, but their business verbs are not, so the emulator does not claim them as room or archive packets;
- there is no high-confidence create/config/admin packet, archive-list transport, prize-claim packet, or retail settlement routine in this client build.

Structurally recovered but semantically unresolved packets:

ID	Direction	Static layout	Proven behavior
0x26B5	C→S	i32 contextId, i32 fieldA, u16 fieldB, i32 fieldC, u16 fieldD	qualifying-screen action
0x26B6	S→C	i8 result; success adds i32 value	success updates screen context
0x26B7	C→S	i32 contextId, i32 selectedEntryId	selects one row from page × 6 + row

ID	Direction	Static layout	Proven behavior
0x26B8	S→C	i8 result	success mutates/removes selected model row
0x26C6	C→S	i32 tournamentId	reachable only for entered participant state 2
0x26C7	S→C	tagged result plus full update or state byte	server-driven model/player-state update

The wiki states the overall tournament supports 64 players. The six-row page and 16-entry final are client facts; connecting them as eight qualifying pages, 48 qualifying matches, and a 64→16 reduction is emulator policy chosen to satisfy the observed views and wiki capacity without inventing new wire fields.

Server implementation

Packet schema

`server-core/src/main/packets/tournament/Tournament.packet` declares the recovered packet IDs, request fields, shared tournament records, and fixed arrays.

Handlers

`game-server/src/main/java/com/jftse/emulator/server/core/handler/tournament/` contains one handler per recovered client request:

- list and join/detail metadata;
- apply and cancel;
- participation information;
- final-bracket definition;
- final-bracket match-state polling.

Success-only tails are written manually where the response is a protocol union: a nonzero first byte terminates the packet, while zero requires a larger body. Modeling only the discriminant in `.packet` avoids auto-writing fields that must not exist on failure.

State model

`TournamentManager` remains the protocol-facing adapter. When Spring services are available it reads authoritative state from `TournamentService`; its old deterministic in-memory data remains only a narrow fallback for isolated protocol tests.

The durable model is deliberately emulator-owned:

Table	Responsibility	Important constraints
<code>TournamentDefinition</code>	format, lifecycle, deadlines, reward	unique title; only 64/16 accepted by creation API
<code>TournamentEnrollment</code>	player, seed, personal state, timestamps	unique tournament/player and tournament/seed

Table	Responsibility	Important constraints
TournamentMatch	stage/round/slot, assigned pair, winner, runtime binding	unique bracket slot, room, and session; optimistic version
TournamentSettlement	immutable prize decision	unique tournament/player/place/product

All four tables are installed by `scripts/sql/tournament.sql` and `scripts/import_sql.sh`. Apply/cancel, seeding, room claims, activation, completion, progression, cancellation, and settlement run inside transaction boundaries with pessimistic reads where a cross-row invariant must be serialized.

Lifecycle, scheduling, and creation policy

`TournamentServiceImpl` validates strict deadline ordering and exposes server-side creation. The scheduler calls `ensureDefaultTournament` and `advanceDueTournaments` every ten seconds. If no active or future definition exists, it creates one recurring 64→16 JFTSE Open Cup, currently configured with product 287×1. Creation is idempotent under concurrent startup.

Lifecycle transitions are:

```
PREPARE → APPLY → PREPARE_QUALIFYING → QUALIFYING
          → PREPARE_FINAL → FINAL → FINISHED
```

`SUSPENDED` and `CANCELED` remain terminal operational states. Exactly 64 enrollments are required when qualifying begins; an underfilled cup is canceled, unfinished matches are aborted, and enrollments are eliminated. Startup recovery removes stale room/session/start bindings and makes fully assigned matches retryable.

Bracket policy and progression

Qualifying pairs seed 1 with 64, 2 with 63, and so on. The durable graph contains 32 round-0 matches and 16 round-1 qualifier matches. Every completed round-0 match fills one side of its target; when both sides are present, that target becomes ready. Completing all 16 qualifier matches marks those winners qualified and moves the tournament to `PREPARE_FINAL`.

Final seeding creates a complete 16→8→4→2→1 tree. Completion is accepted only when stage, match status, room ID, game-session ID, reporter, and claimed winner all match the locked durable assignment. Concurrent duplicate completions cannot advance twice.

Room, relay, spectators, and teardown

Tournament rooms use the explicit compatibility convention `T#<tournamentId>`. Creation resolves that player's current persisted match in the tournament and forces a public, two-player Basic room. The assigned pair occupy player positions, first creator becomes master, and assignment cannot be changed by normal room mutation handlers. Start requires exactly the two assigned participants and the non-master to be ready.

The game session is persisted before relay handoff. Both participants must connect within 30 seconds; spectators neither count toward nor block readiness. Ordinary room positions above 3 can contain spectators before start, but this does **not** prove retail spectator protocol and does not add admission to an already running match.

Normal leave, disconnect, failed startup, relay failure, rejected completion, and timeout all use central game-session teardown. Teardown cancels scheduled work, clears every client's session ID, clears the session roster, removes the session, and returns the durable match to a retryable state where appropriate.

Archives and prize settlement

Finished definitions remain queryable through the durable archive service/manager API, while canceled definitions remain persisted in the main durable listing. No native archive-list request was recovered, so no invented client packet is exposed.

The emulator awards first place by inserting `TournamentSettlement` and adding the configured product to inventory in one transaction. The unique settlement key makes retries idempotent; an inventory failure rolls the settlement back. Reward rendering proves only display in retail—the award behavior here is emulator policy.

Red/green development record

Red 1: no tournament handlers

The development server accepted the client connection but did not answer the list request. The tournament screen contained no row.



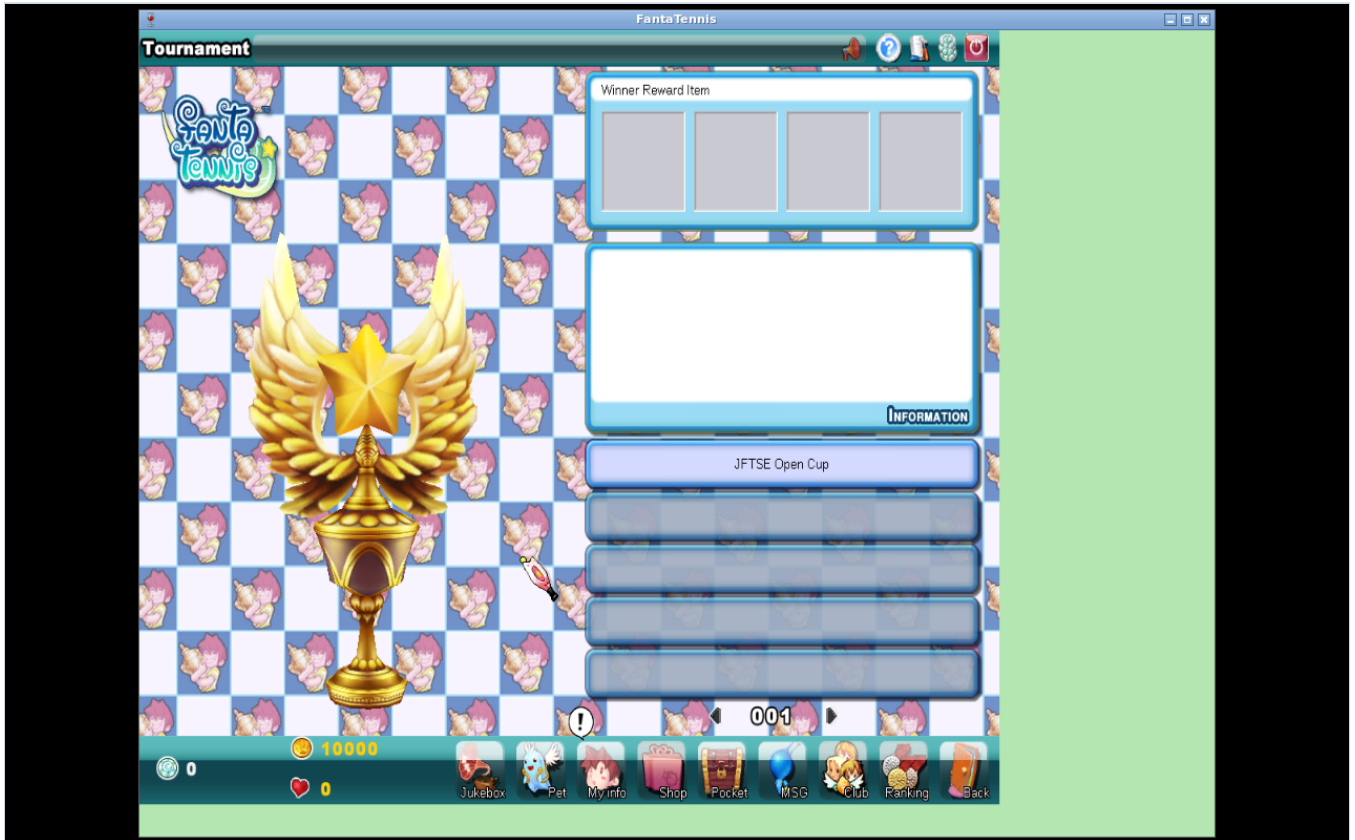
Artifacts:

- `client-server-baseline.pcap`
- `game-server-no-tournament-handler.log`

- `maven-tournament-tests-before-implementation.log`

Red 2: list existed, detail response was missing

Selecting the row produced `0x26AD` and `0x26BE`, but the screen could not populate correctly without both metadata and player information responses.





Red 3: 0x26AE omitted mandatory success metadata

Returning only a success byte was not sufficient. The client needed the tournament metadata block after the successful status.



Red 4: final bracket request unhandled

Clicking the enabled final bracket button revealed `0x26C0` with a six-byte payload.



Red 5: definition accepted, automatic match request unhandled

A structurally accepted `0x26C1` caused the client to send `0x26C2` automatically. Without `0x26C3`, the detail screen remained visible.



Red 6: correct count, invalid record state

Fifteen `00 00` records satisfied `got=15`, `expected=15`, but the parser still took its failure return. This was the critical proof that cardinality alone was insufficient.



Artifacts:

- `bracket-zero-state-rejected.pcap`
- `client-c3-parser-trace.txt`
- `game-server-bracket-complete.log`
- `maven-tournament-bracket-match-before-implementation.log`

Green: exact packet contract and real-client flow

Focused final test result:

```
Tests run: 21, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Full Java 21 reactor result after merging current development:

```
Tests run in game-server: 28, Failures: 0, Errors: 0, Skipped: 0  
All 11 reactor modules: SUCCESS  
BUILD SUCCESS
```

Continuation validation after adding the durable engine:

```
Focused protocol/service/scheduler/room/session suite: BUILD SUCCESS  
MariaDB persistence/concurrency suite: 15 tests, 0 failures/errors  
Full reactor: 59 tests, 0 failures, 0 errors, 4 skipped  
Package: BUILD SUCCESS
```

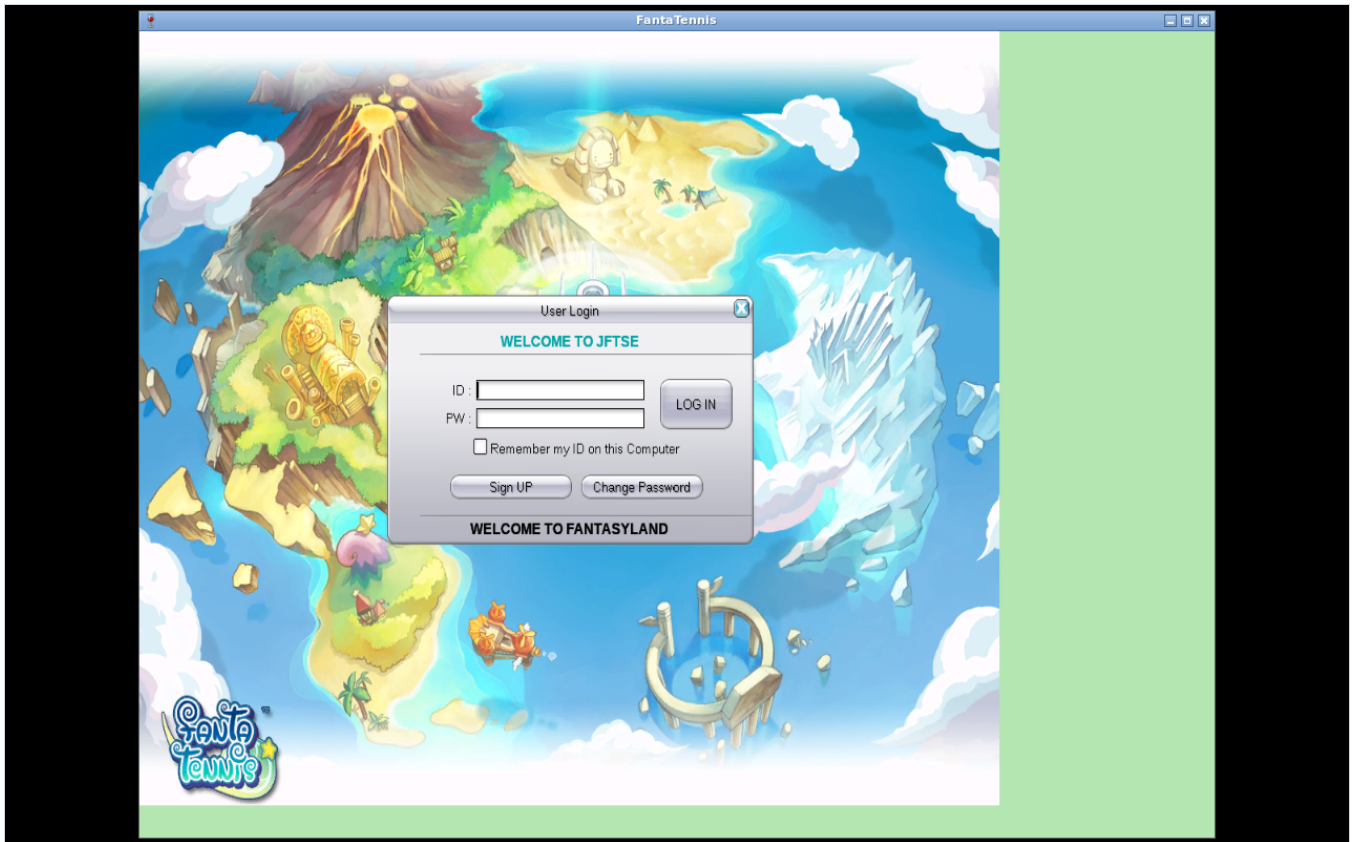
The MariaDB suite includes concurrent 65-player enrollment, concurrent qualifying seeding, concurrent final completion, restart recovery, progression, rollback, and exactly-once settlement. The packaged `game-server.jar` was also restarted with a synthetic ACTIVE match; its stale `roomId`, `gameSessionId`, and `startedAt` were cleared and status returned to READY.

Artifacts:

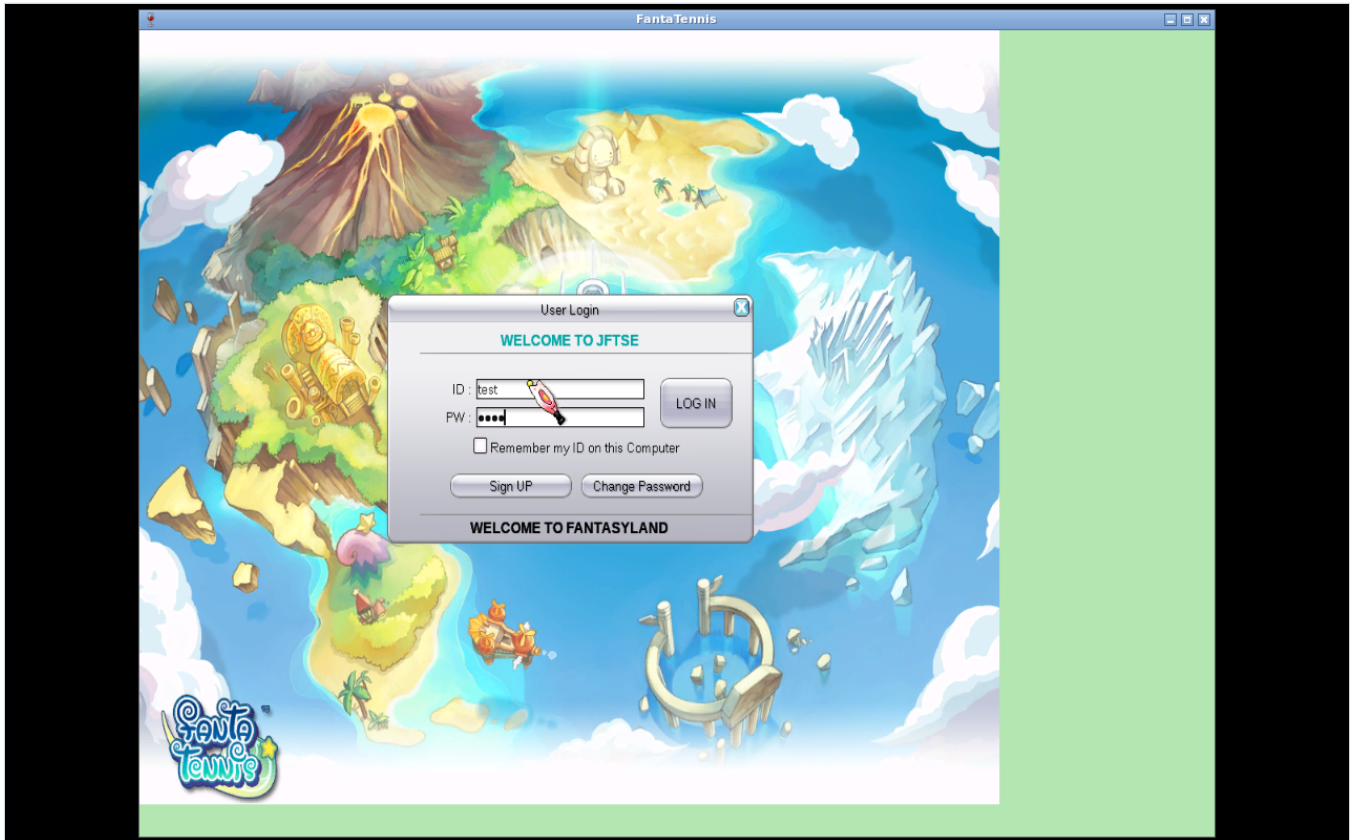
- `maven-tournament-final-review-fixes.log`
- `maven-full-reactor-tests-final.log`
- `maven-full-reactor-continuation.log`
- `packaged-restart-before-after.txt`

Real-client end-to-end visual evidence

1. Client reached the local authentication server



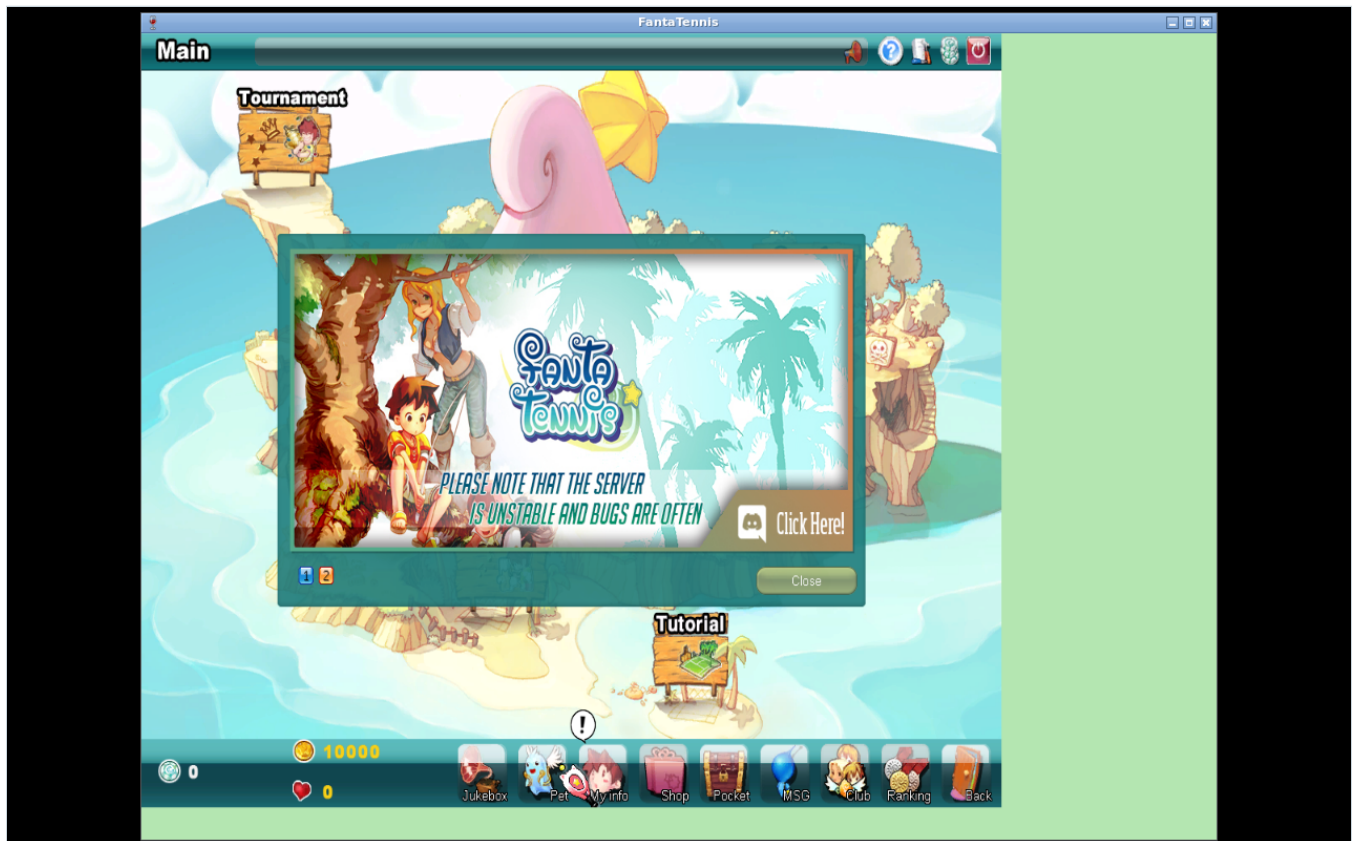
2. Test credentials were entered



3. Test character and local channel were selected

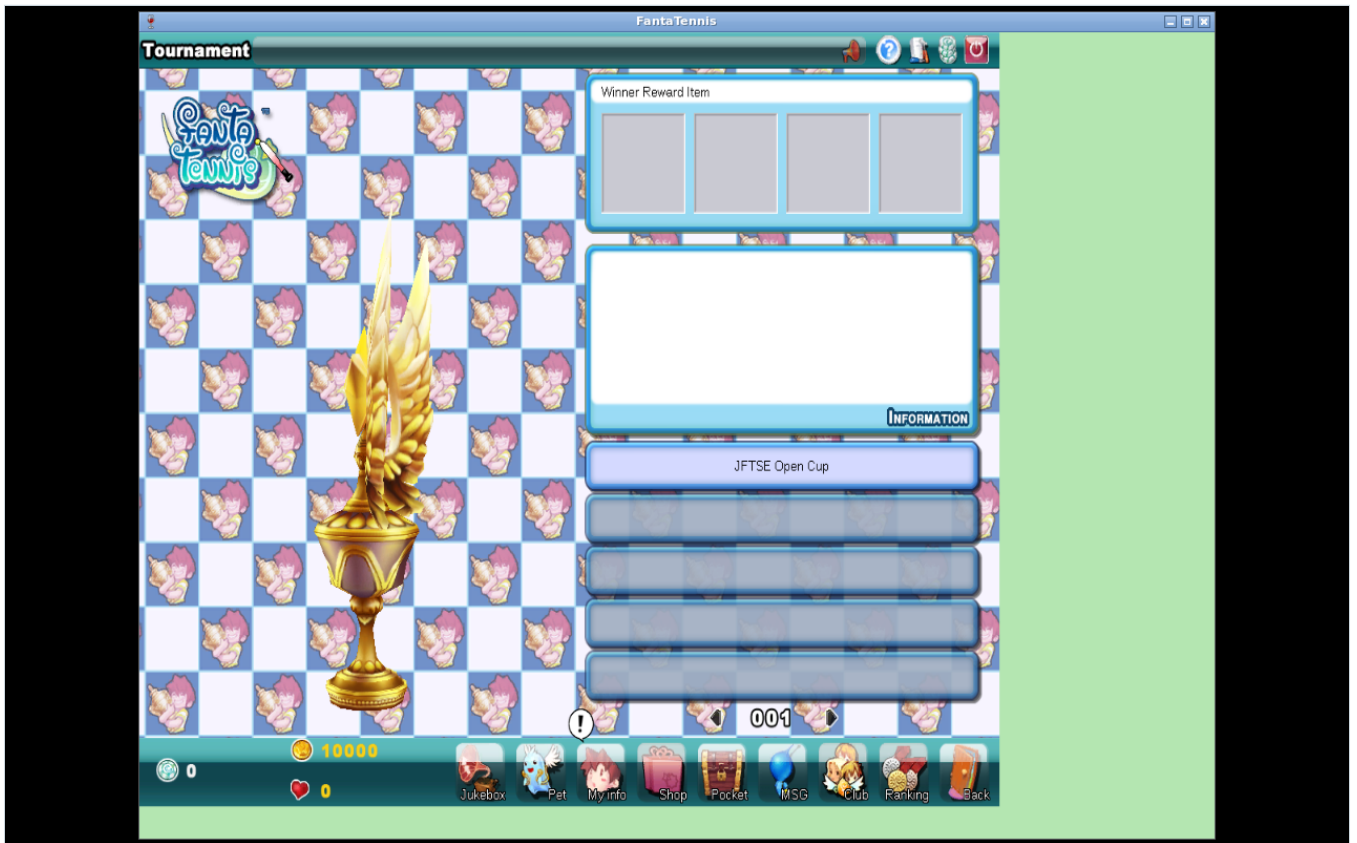


4. Client connected to the local game server



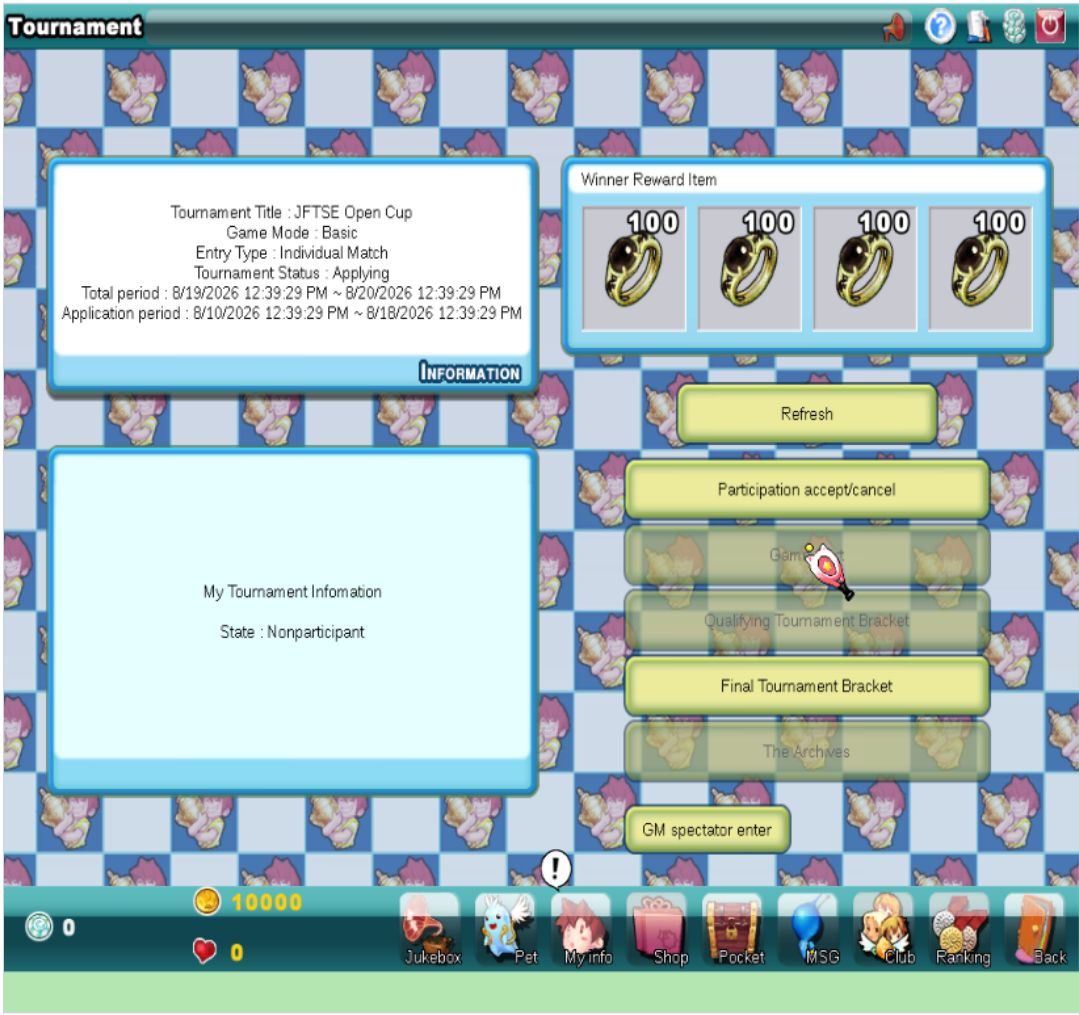
5. Tournament list rendered

The deterministic tournament appears as `JFTSE Open Cup`; the client also renders its page control and tournament art.

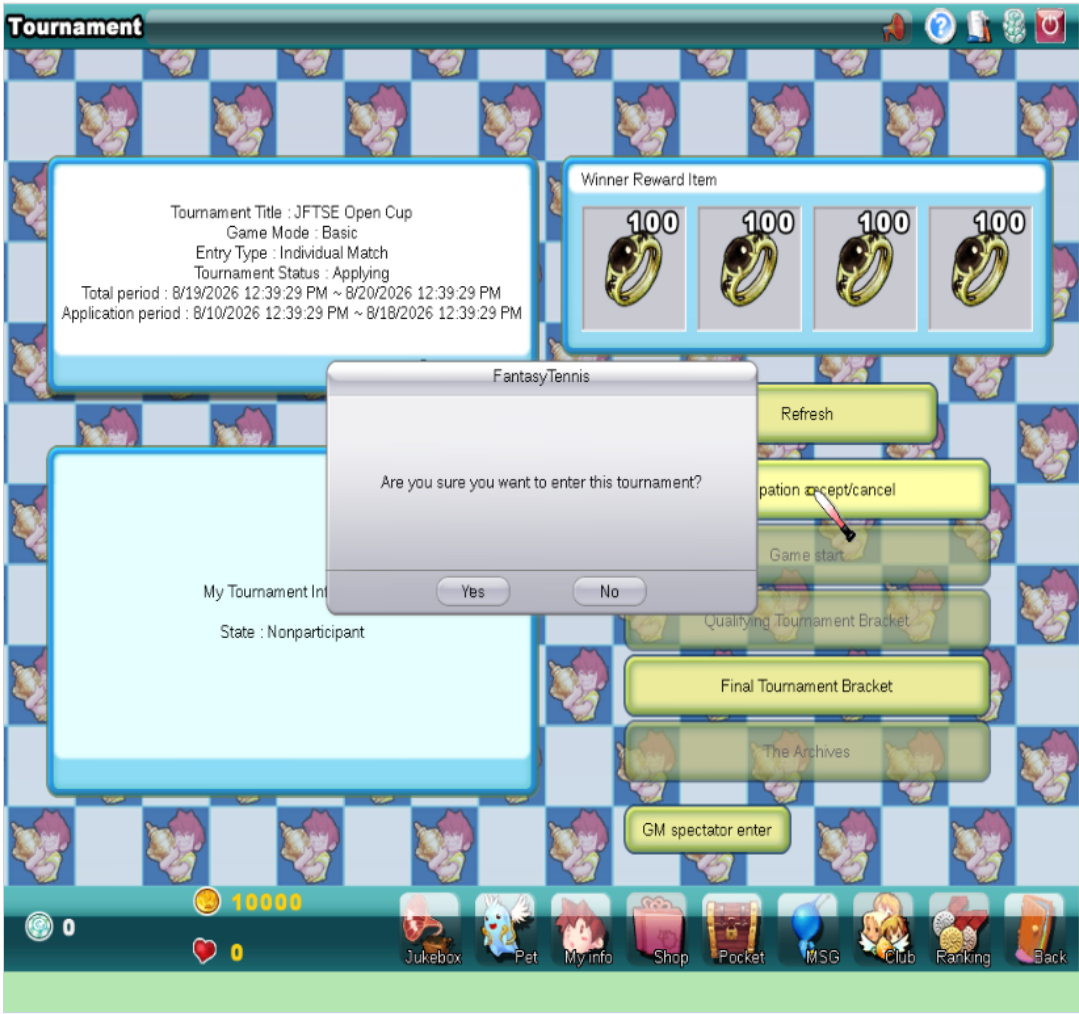


6. Tournament detail and winner rewards rendered

The client displays title, Basic game mode, Individual Match entry type, application dates, status, and winner reward items.



7. Apply confirmation



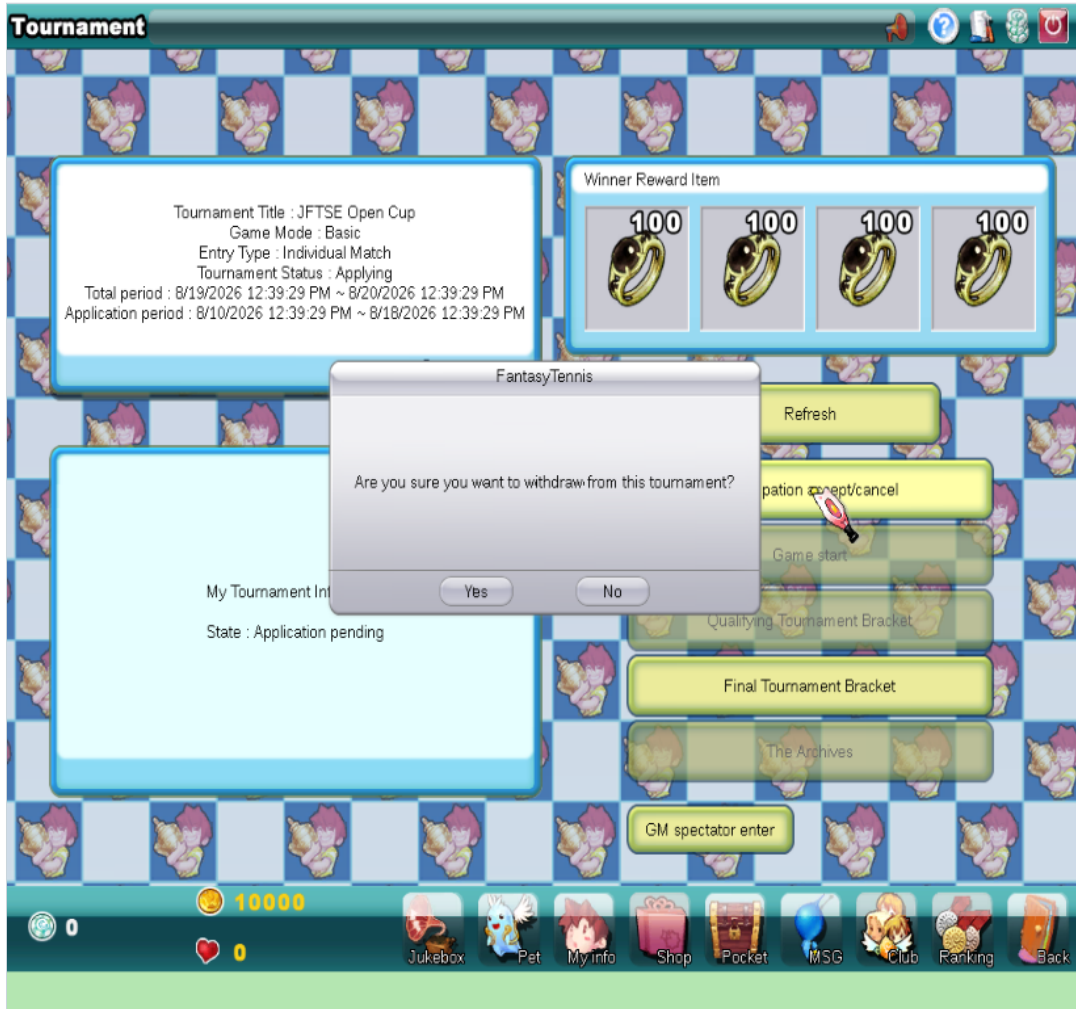
8. Apply success

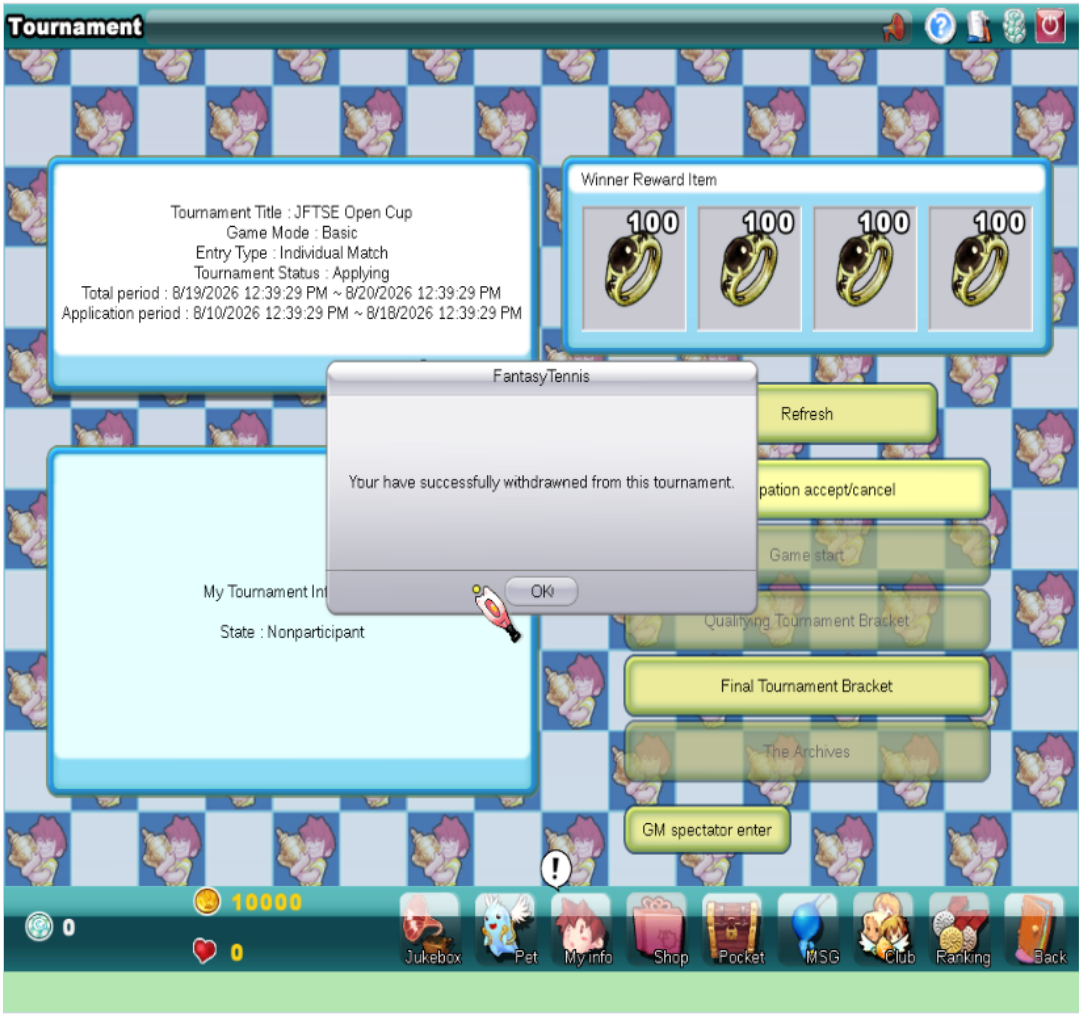


9. Applied state persisted across detail refreshes

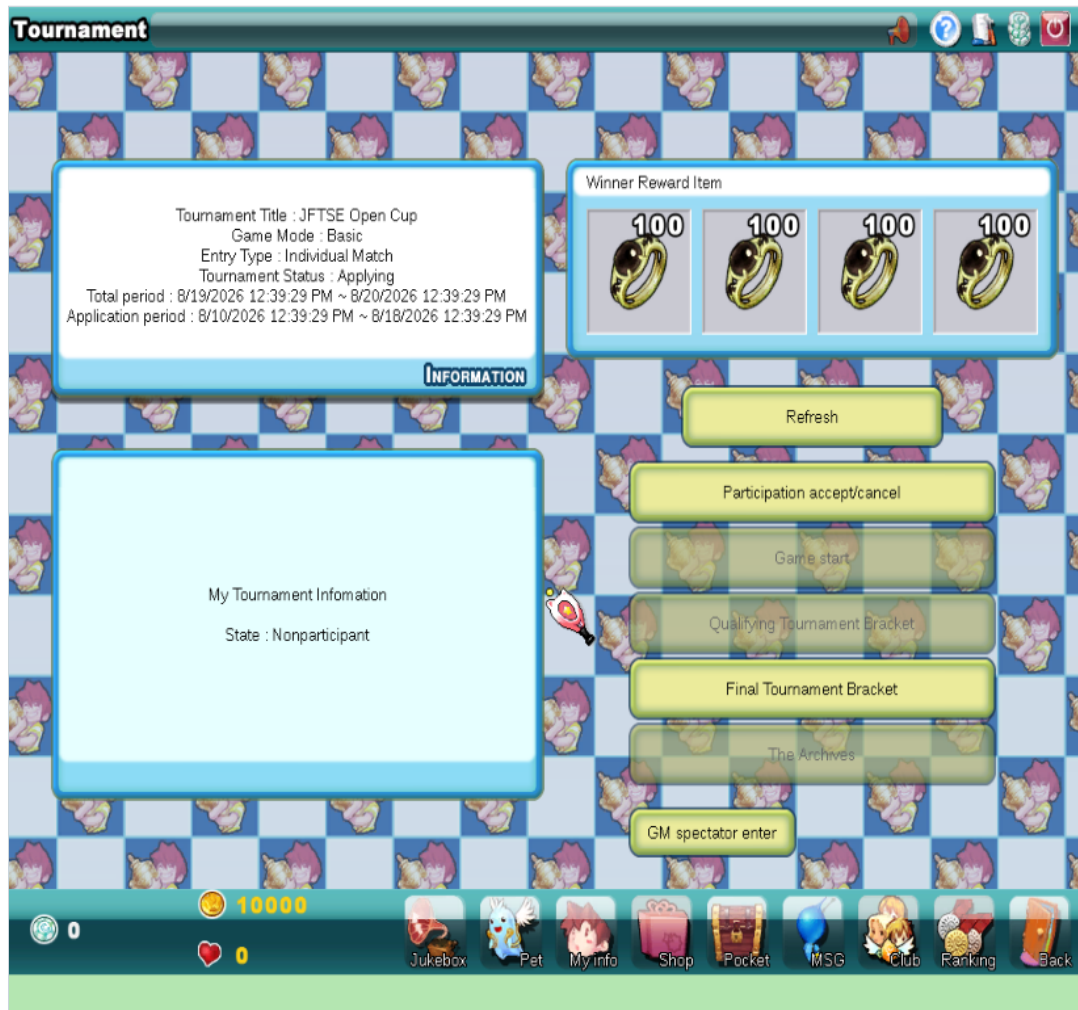


10. Cancel confirmation and success



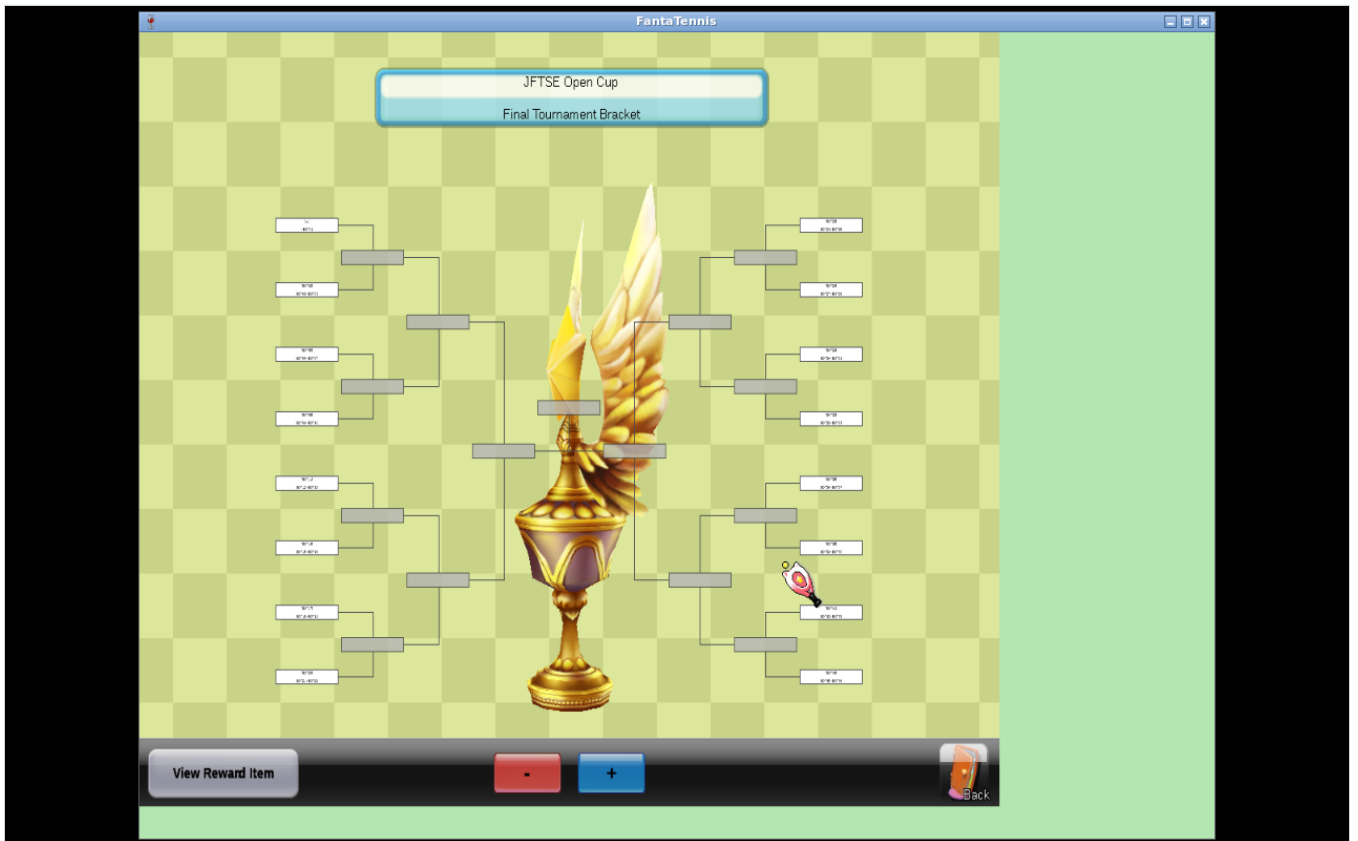


11. Nonparticipant state restored

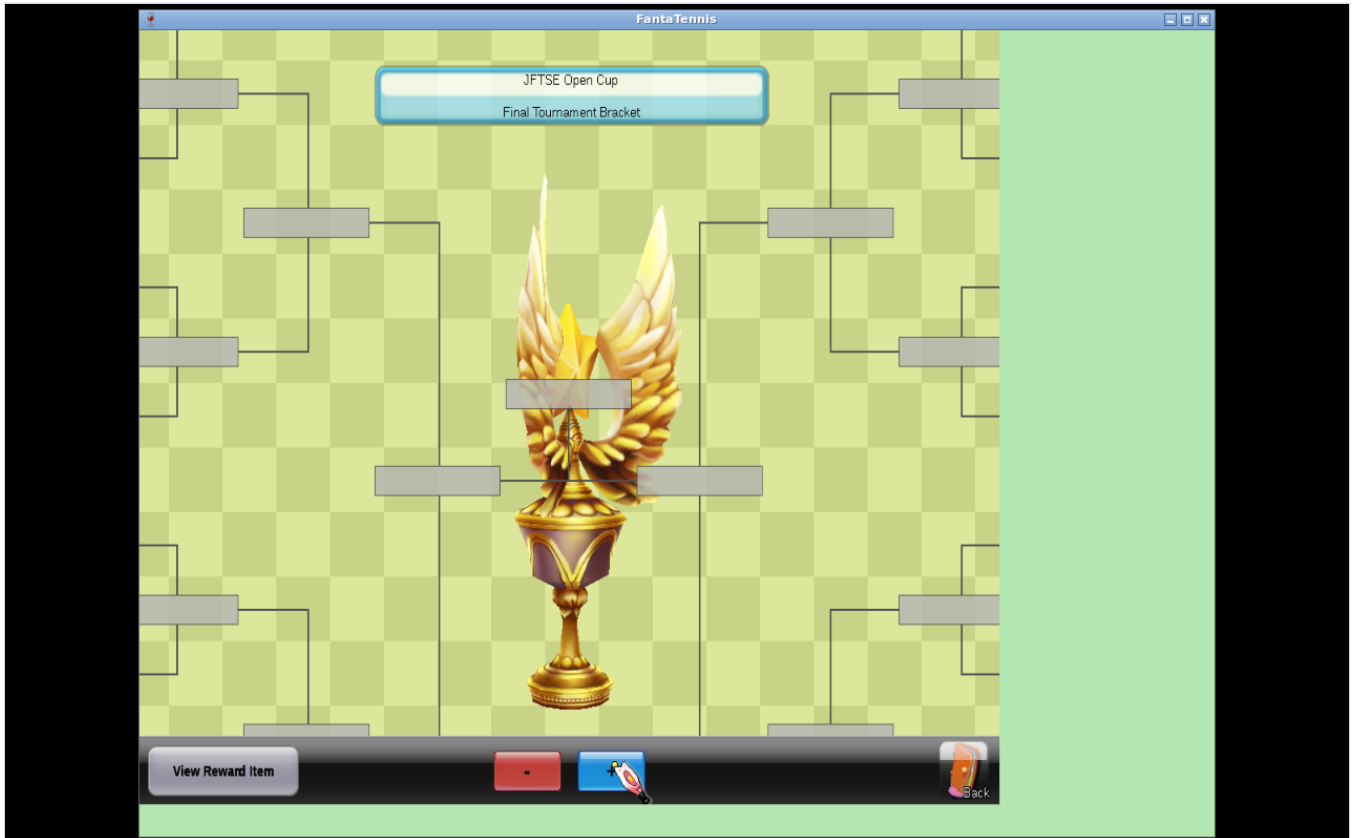


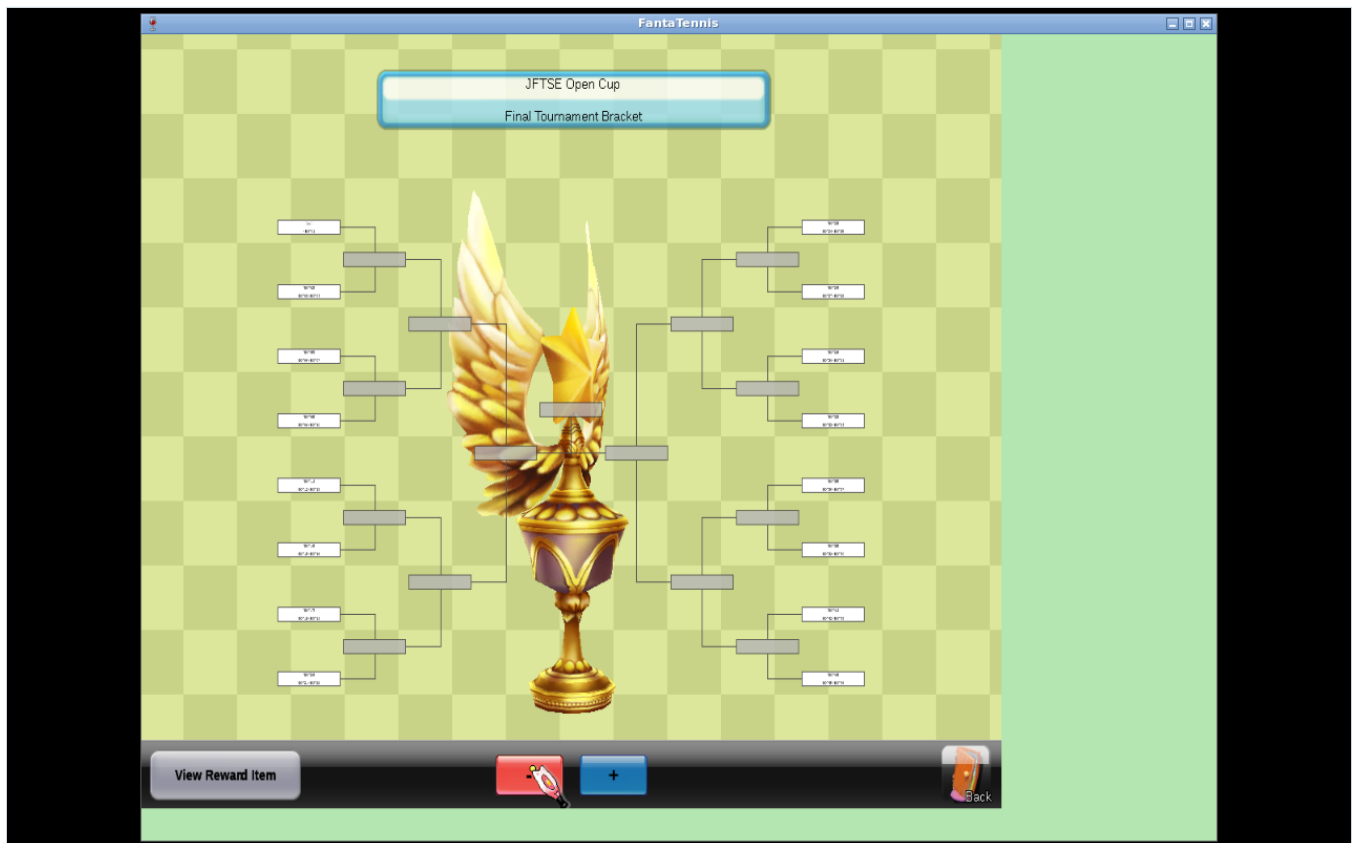
12. Final bracket visibly rendered

This is the decisive UI proof. It corresponds to the fresh PCAP containing `0x26C0` , `0x26C1` , automatic `0x26C2` , and accepted `0x26C3` , plus the runtime `c3_ok` trace.



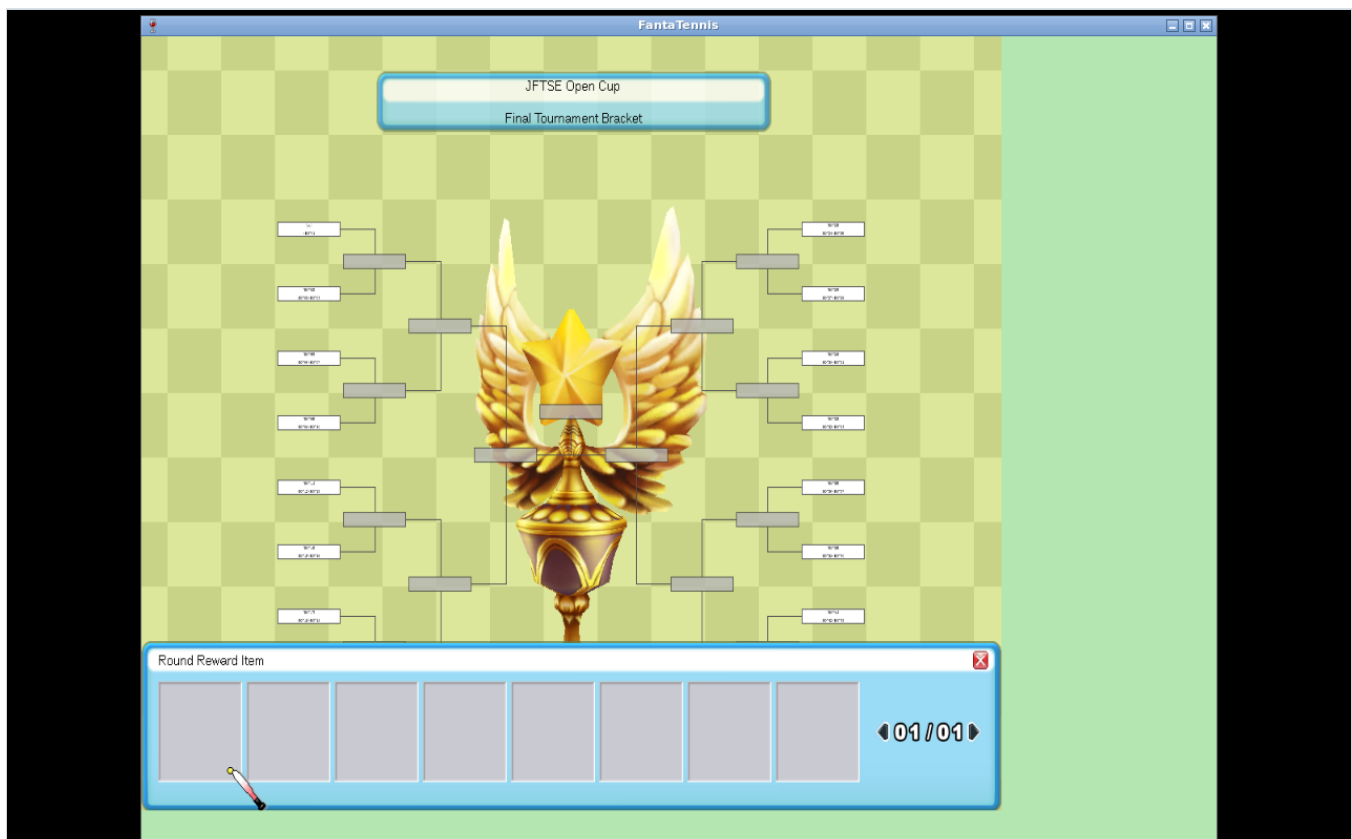
13. Zoom in and out controls worked

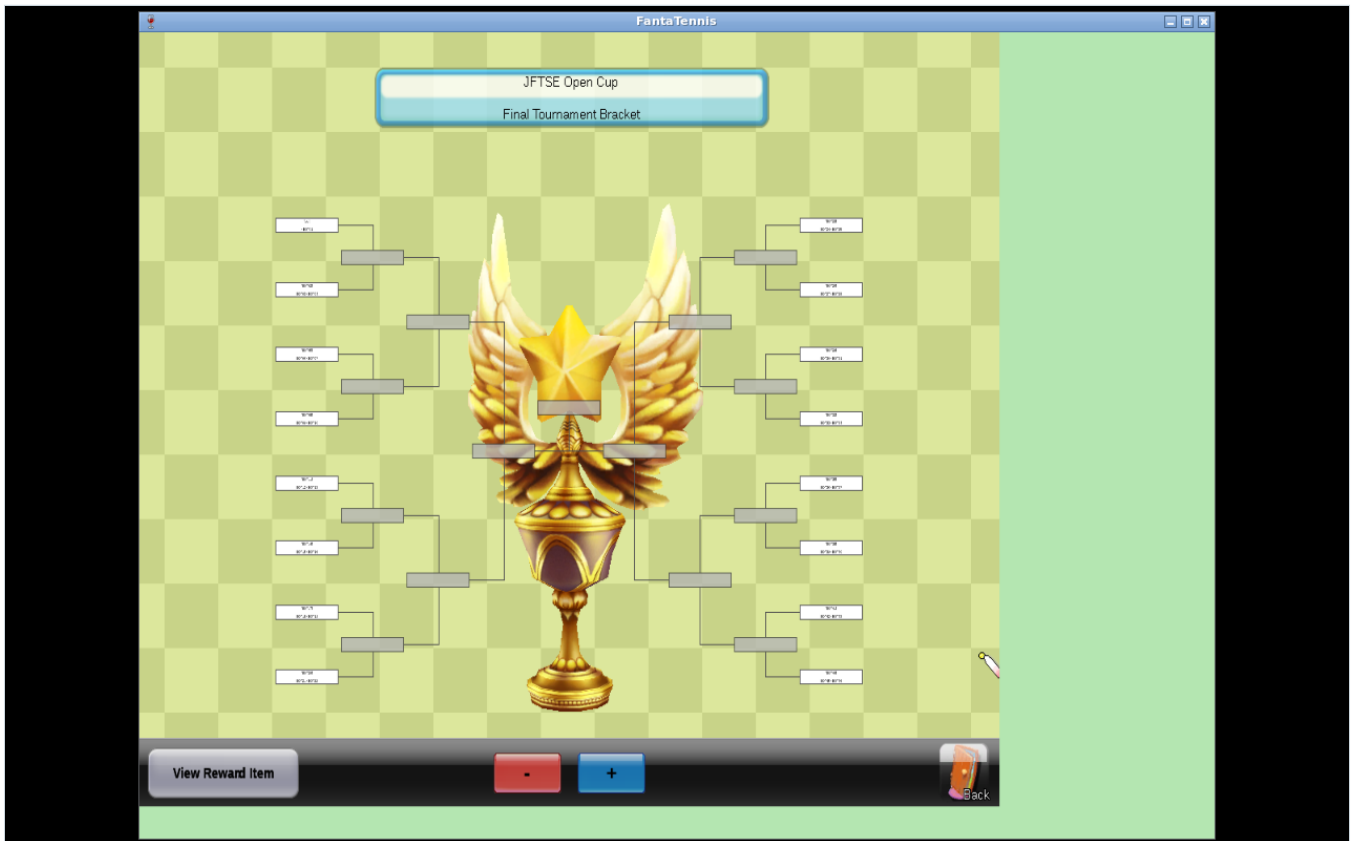




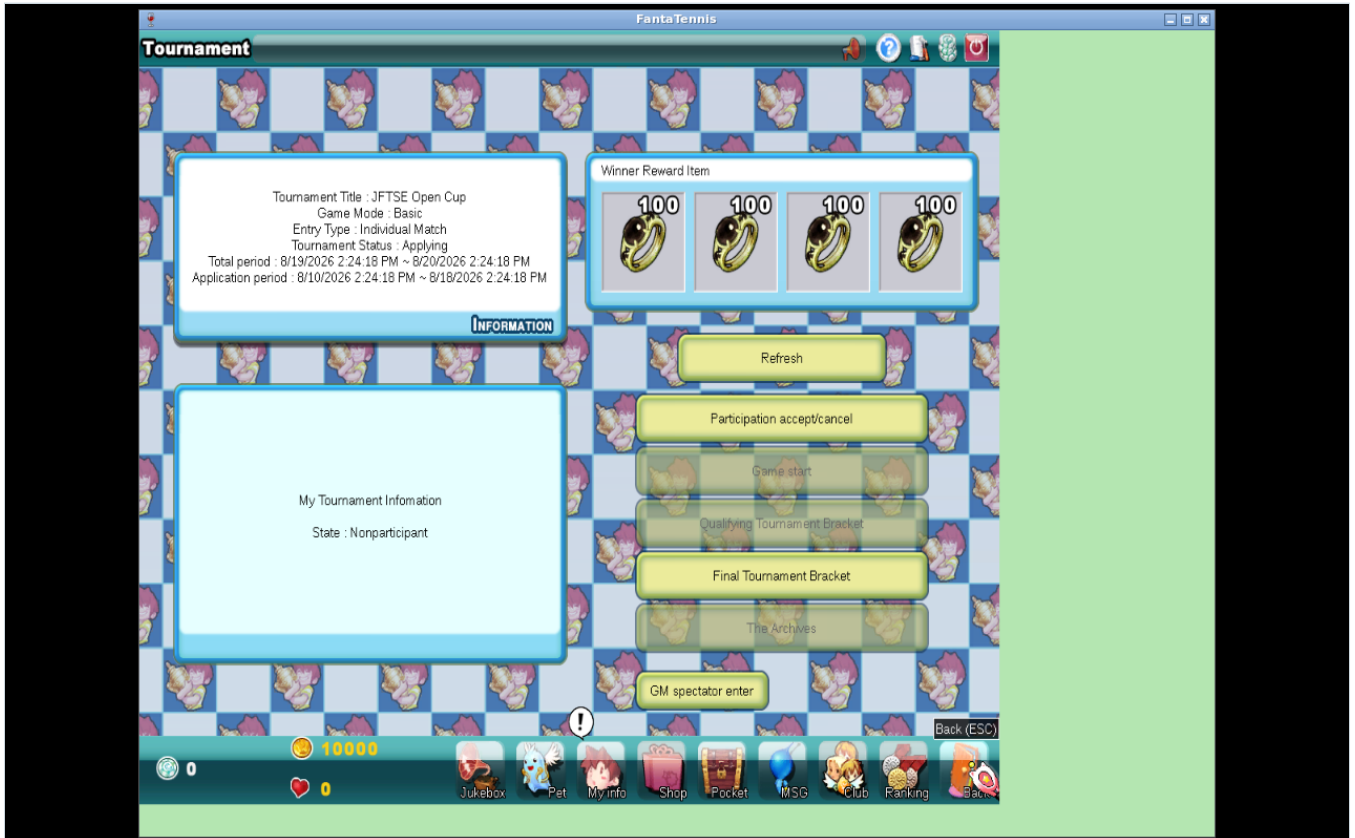
14. Round reward control opened and closed

The panel is empty because no round-progression rewards were invented. The already recovered winner rewards remain visible on the detail screen.





15. Back navigation returned to tournament details



Native two-client tournament match continuation

Two isolated, unmodified clients were run on separate Wine prefixes and Xvfb displays with accounts `NativeOne` and `NativeTwo`. The fixture was server-created and used only player IDs and the durable bracket graph; no client packet was injected.

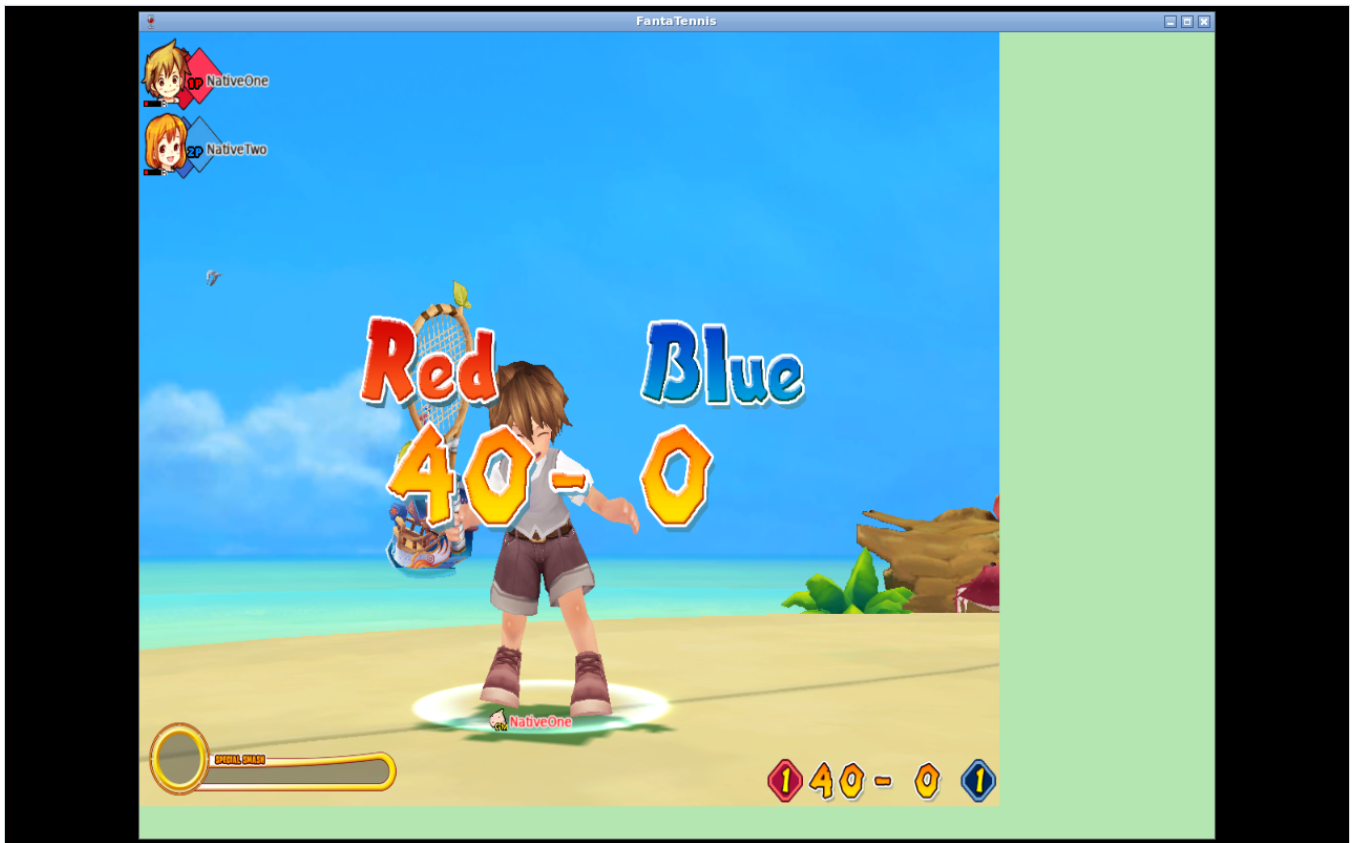
16. Assigned players joined a locked tournament room

`NativeOne` created `T#2`. The server resolved match 2, forced the title `Tournament 2-Q1-1`, Basic mode, two participant slots, and immutable assignment. `NativeTwo` saw the ordinary lobby row and joined the second player position.



17. Ready/start and relay handoff succeeded

The guest readied, the master sent native `CMSGStartGame (0x177B)`, and both clients sent `CMSGConnectedToRelay (0x03F3)`. Relay decoded both clients on the same session, each with `isSpectator=false` and the same assigned player set.



The durable transition captured around start was:

```
READY: status=1, roomId=0, gameSessionId=NULL, startedAt=NULL
ACTIVE: status=2, roomId=0, gameSessionId=53363, startedAt=<timestamp>
```

Room ID 0 is valid in the existing room allocator; SQL `NULL`, not zero, means unbound.

18. Completed native match advanced the durable bracket

The Basic match ran to its ordinary client-reported completion. The service accepted completion only for the persisted room/session/participants, marked the source match complete, eliminated the loser, and placed the winner into the next qualifying slot. The before/after snapshot is preserved in `native-match-db-before-after.txt`, with decisive game and relay events in `native-match-protocol.log`.



This proves multi-client room creation, assigned admission, readiness, game start, relay registration, ordinary Basic gameplay, durable ACTIVE state, and bracket advancement for the emulator. It does **not** prove retail used `T#<id>` room names or the same completion policy.

19. Native championship settled the durable first-place prize once

A separate valid 16-entry final graph left only round 3/slot 0 READY, assigned to `NativeOne` and `NativeTwo`. The two unmodified clients created and joined `Tournament 3-F4-1`, entered relay session 28229, and played the championship through the normal Basic match-end path.



The durable transition was:

```

before: tournament=FINAL, match=READY, settlement rows=0,
        configured prize absent, pocket belongings=0
active: match=ACTIVE, roomId=0, gameSessionId=28229
after:  tournament=FINISHED, match=COMPLETED, winner state=4,
        settlement rows=1, SPECIAL/itemIndex 1/Durable count=1,
        pocket belongings=1

```

The configured product index was 287; the database resolves it to the durable SPECIAL item above. The match, settlement, and exact inventory before/after rows are preserved in `native-final-db-before-after-restart.txt`. `native-final-protocol.log` correlates the start, both relay registrations, terminal point, accepted completion, and packaged restart. The credential-free trimmed `native-final-match.pcap` contains only the two game and two relay connections for this match.



The exact packaged jar, SHA-256 `78692b9d3e49d350824e49b557458957ecfcc42a3c4ae263f425dd95028bc7e8`, was restarted afterward. The finished match, one settlement, one item, and one belonging persisted without duplication. The restart proves durable retention/no startup duplicate; the MariaDB concurrent-completion test separately proves that two completion attempts produce one `COMPLETED`, one `ALREADY_COMPLETED`, and one inventory grant.

Network and runtime evidence

The complete file-integrity manifest is `SHA256SUMS`. It excludes itself so the recorded hashes remain reproducible.

Primary captures

Artifact	Purpose
client-server-green.pcap	Original list/detail/apply/cancel green session
tournament-bracket-sentinel-green.pcap	Fresh 328-frame final bracket session with accepted FF 00 states
native-final-match.pcap	Credential-free two-client championship game/relay traffic
tournament-complete-packet-exchange.tsv	Human-readable tournament packet index extracted from the fresh capture
tournament-bracket-sentinel-packets.tsv	Exact first C0/C1/C2/C3 exchange
client-bracket-parser-success-trace.txt	Count, comparison, and repeated client c3_ok probes
game-server-bracket-sentinel-green.log	Server-side decoded and encoded tournament traffic

First accepted bracket exchange

Frame	Direction	ID	Total TCP bytes	Key values
209	C→S	0x26C0	14	tournament 1, type 1, page 0
211	S→C	0x26C1	594	success, 16 fixed rows
212	C→S	0x26C2	14	tournament 1, type 1, index 0
213	S→C	0x26C3	47	success, count 15, FF 00 × 15

The client repeated 0x26C2 approximately every ten seconds. Every captured poll received the same 47-byte response and every instrumented parse took c3_ok.

Reproduction

Java validation

Use Java 21:

```

export JAVA_HOME=/tmp/jdk21
export PATH="$JAVA_HOME/bin:$PATH"

TOURNAMENT_TESTS=\
'TournamentBracketMatchPacketHandlerTest,'\
'TournamentBracketPacketHandlerTest,'\
'TournamentJoinPacketHandlerTest,'\
'TournamentManagerTest,'\
'TournamentPacketProtocolTest,'\
'TournamentInfoPacketHandlerTest,'\
'TournamentRoomCoordinatorTest,'\
'TournamentSchedulerTest,'\
'TournamentServicePersistenceTest,'\
'GameSessionManagerTest'

mvn -pl game-server -am \
  -Dtest="$TOURNAMENT_TESTS" \
  -Dsurefire.failIfNoSpecifiedTests=false \
  test

mvn test
mvn -pl game-server -am -DskipTests package

```

PCAP extraction

```

sudo tshark \
  -r docs/tournament-mode/evidence/green/client-e2e/tournament-bracket-sentinel-green.pcap \
  -Y 'tcp.port == 5895 && tcp.len >= 8' \
  -T fields \
  -e frame.number -e frame.time_relative \
  -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.len -e tcp.payload \
  | grep -i -E 'ad26|ae26|af26|b026|b126|b226|b326|b426|be26|bf26|c026|c126|c226|c326'

```

Runtime client parser verification

The Wine client was run under Xvfb/Openbox. Linux tracefs uprobes were attached to the client executable at parser branches recovered from disassembly. The PE mapping used for this build was:

```
probe file offset = virtual address - 0x00400000
```

The decisive probes were:

```

0x5c9dcd  derived count/object state
0x5c9deb  received-versus-expected count comparison
0x5calc0  success return
0x5calcd  failure return

```

Probe offsets are build-specific and must not be reused against another executable hash.

Tests added

The test suite covers:

- decoding every recovered request field;

- exact packet IDs and little-endian payload order;
- success-only response tails;
- terminal one-byte failures;
- five reward records without a prefix;
- both fixed 16-pair arrays without prefixes;
- rejection of wrong fixed-array cardinality;
- full 16-row C1 serialization;
- exact 15-record `FF 00` C3 success payload;
- qualifying pages 0–7 with six entries and page rejection outside that range;
- durable per-player apply/cancel isolation, duplicate and capacity rejection;
- 64→16 seeding, all qualifying/final rounds, and completed-match bracket advancement;
- lifecycle deadlines, underfilled cancellation, archives, and recurring default creation;
- room claim/admission, immutable assignments, readiness, spectators, and mutation rejection;
- relay activation, timeout, leave/disconnect/failure teardown, and restart recovery;
- concurrent 65th-player admission, concurrent seeding, and concurrent completion;
- inventory prize grant, settlement idempotency, and transaction rollback on grant failure;
- unknown tournament and stale room/session/reporter/winner rejection.

Known limitations and next evidence required

The requested emulator capabilities are implemented. These remaining items are retail-evidence or trust-boundary gaps and must not be erased by implementation claims:

1. **Type-0 row semantics:** static RE proves six visible fixed-width rows per page, but not the retail meaning of each of the three strings or final-page padding behavior.
2. **B5-B8 business verbs:** request/response layouts are recovered, but the selected-row mutation and success transition are not identified confidently enough to expose.
3. **C6/C7 relationship:** C6 is participant-only and C7 is a server update; native timing/dataflow still needs to identify the retail action and whether C7 is its response.
4. **Retail room handoff:** `T#<tournamentId>`, ordinary room creation, 30-second timeout, and first-place-only settlement are compatibility policy, not original-server facts.
5. **Archives:** durable records and a service query exist, but this client has no recovered archive-list transport or enabled archive walkthrough.
6. **Spectator admission:** spectators in pre-start room positions are supported. Native admission to an already running match remains unsupported and unproven.
7. **Completed C3 rendering:** persisted result/state records are supplied, but the full retail visual semantics of every nonempty C3 value remain incompletely characterized.
8. **Rewards:** winner-reward descriptors prove display only. The emulator's exactly-once inventory award is policy; retail claim/award transport is unknown.
9. **Result authority:** the emulator validates assignment, room, session, reporter, and winner membership, but still consumes the existing client-reported match-end/score path. A server-authoritative tennis simulation is outside this work.
10. **Native administration:** no create/config/admin request exists with high confidence. Custom creation is intentionally server-side rather than an invented client protocol.

Conclusion

The development branch previously exposed a Tournament entry point with no supporting server protocol. The combined work now drives the real client through listing, details, participation, qualifying/final views, assigned room creation and join, readiness, relay handoff, two-client Basic matches, durable bracket advancement, championship completion, and inventory settlement. MariaDB concurrency tests cover the complete 64→16→winner lifecycle, restart recovery, archives, and exactly-once prize policy.

The result is a complete emulator tournament engine for the selected 64/16 Basic format, not a claim that every original-server business rule has been recovered. Native facts, static facts, compatibility choices, and remaining unknowns are kept separate so future captures can replace policy without rewriting history.